

Lab4 Report: Conditional VAE for Video Prediction

張苙烜/ 資科工碩 313551079

1 Introduction

在這個 lab 中，需要去實作一個 VAE-based model Conditional Variational Autoencoder (CVAE)，輸入上一幀的 image X_{t-1} 和當前的 pose p_t 去預測當前的 image $\hat{X}_t$ 。透過調整 teacher_forcing_ratio 和 KL annealing strategy 和 scheduling 試圖讓 X_{t-1} 和 $\hat{X}_t$ 最接近，提升模型的收斂速度與生成品質，最後透過 psnr 評估最終結果。模型分為兩個部分，需要將輸入經過 encoder 之後轉成 latent space 參數，再根據這個分布抽樣，再透過 decoder 和 generator 生成新的影像。訓練中有 MSE loss 和 KL loss，前者衡量 predict 和 ground truth 的差距，後者是評估 latent space 和標準常態分布的差距。

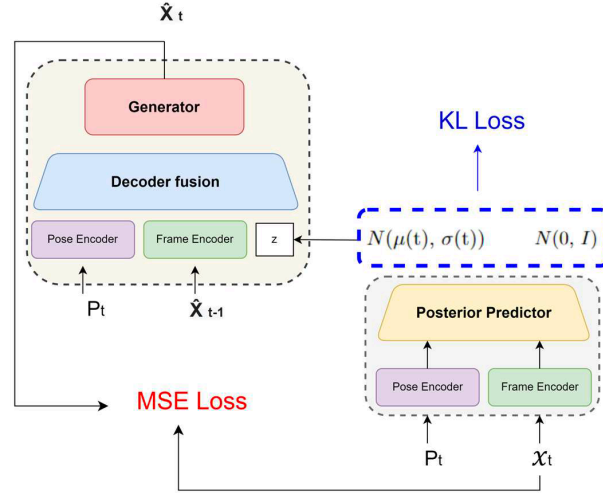


Figure 1: flow chart

2 Implementation details

2.1 training/testing protocol

遍歷影片所有的 frames，將 current_frame 和 current_label 輸入透過 encoder 轉成特徵向量，將向量丟入 Gaussian_Predictor 輸出 latent space variable $\mu, \log \sigma^2$ 後，透過 reparameterization 轉換抽樣出 z 。將上一幀的 previous_feature(X_{t-1}) 和 label_feature(P_t) 輸入 decoder 再由 generator 生成 predict_frame。再來計算 loss，kl_criterion 計算 KL divergence，藉由 beta 決定 KL loss 的權重大小，再加上 mse loss，公式如下：

$$\mathcal{L} = \text{MSEloss} + \beta \cdot \text{KLloss} \quad (1)$$

```
1 def training_one_step(self, img, label, adapt_TeacherForcing):
2     # TODO
3     self.train()
4     self.optim.zero_grad()
5
```

```

6     B,T,C,H,W = img.shape
7     total_loss = 0.0
8
9     previous_frame = img[:, 0, :, :, :].clone()
10
11     for t in range(1, T):
12         current_frame = img[:, t, :, :, :].clone()
13         current_label = label[:, t, :, :, :].clone()
14
15         # feature extraction
16         image_feature = self.frame_transformation(current_frame)
17         label_feature = self.label_transformation(current_label)
18
19         # turn to z latent space
20         z, mu, logvar = self.Gaussian_Predictor(image_feature, label_feature)
21
22         previous_feature = self.frame_transformation(previous_frame)
23         decoder_out = self.Decoder_Fusion(previous_feature, label_feature, z)
24         predict_frame = self.Generator(decoder_out)
25
26         # loss calculation
27         kl_loss = kl_criterion(mu, logvar, B)
28         beta = self.kl_annealing.get_beta()
29         step_loss = self.mse_criterion(predict_frame, current_frame) + beta * kl_loss
30         total_loss += step_loss
31
32         if adapt_TeacherForcing:
33             previous_frame = current_frame
34         else:
35             previous_frame = predict_frame.detach()
36
37         # update the model
38         total_loss.backward()
39         self.optimizer_step()
40     return total_loss / (T - 1) # return the average loss of the batch

```

下面是 testing 的 protocol，與 training 邏輯相像，只是在 z 中需要隨機取樣去 inference

```

1     for i in range(1, 630):
2         previous_frame = self.frame_transformation(decoded_frame_list[i-1].to(self
3             .args.device))
4         current_label = self.label_transformation(label[i]).to(self.args.device)
5         z = torch.randn((1, self.args.N_dim, 32, 64), device=self.args.device)
6         decoded = self.Decoder_Fusion(previous_frame, current_label, z)
7         decoded = self.Generator(decoded)
8         decoded_frame_list.append(decoded.cpu())
9         label_list.append(current_label.cpu())

```

2.2 reparameterization tricks

模型輸出了 μ 和 $\log\sigma^2$ ，這邊模型預測 $\log\text{var}$ 而不預測 var 本身是因為，預測 var 會有負數的問題。最後回傳的 z 加入常態分布的 noise，轉換成可微分的 z ，有辦法在後續計算梯度更新學習。

```

1     def reparameterize(self, mu, logvar):
2         std = torch.exp(0.5 * logvar)
3         eps = torch.randn_like(std) # eps = N(0,1)
4         return mu + eps * std # z = mu + sigma * eps

```

2.3 teacher forcing strategy

Teacher Forcing Ratio 決定了再訓練階段是否要使用 ground truth 當作輸入，並且怎麼去遞減這個 ratio。分別有三個參數 `tfr`, `tfr_sde`, `tfr_d_step` 決定起始的 ratio 和多少個 epoch 之後要降低，最後是每次需要將低多少。這邊使用線性遞減，但要注意不可以小於 0。

```
1 def teacher_forcing_ratio_update(self):
2     # TODO
3     if self.current_epoch >= self.tfr_sde:
4         self.tfr = max(0.0, self.tfr - self.tfr_d_step)
```

2.4 KL annealing ratio

這項作業實驗了三個 annealing ratio, Cyclical, Monotonic 和 Without (set beta = 1)。Monotonic 是從 0 到 1 線性上升; Cyclical 將訓練期間切成 `n_cycle` 個週期，在每個週期中， β 值會從 0 緩慢線性增加至 1，遞增的長度由 ratio 控制，接著立即回落為 1，維持到下一個週期再重新上升，以此實現 Cyclical， β 變化如下圖 (Figure 2)。

```
1 def get_beta(self):
2     if self.kl_anneal_type == 'Monotonic':
3         return min(1.0, self.current_epoch / self.args.num_epoch)
4     elif self.kl_anneal_type == 'Cyclical':
5         return self.beta_list[min(self.current_epoch, len(self.beta_list) - 1)]
6     elif self.kl_anneal_type == 'Without':
7         return 1.0
8
9 def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0, n_cycle=1, ratio=1):
10    L = np.ones(n_iter) * stop # create a list of length n_iter initialized with
11    stop value
12    period = n_iter / n_cycle
13    step = (stop - start) / (period * ratio)
14
15    for c in range(n_cycle):
16        v, i = start, 0
17        while v <= stop and int(i + c * period) < n_iter:
18            L[int(i + c * period)] = v
19            v += step
20            i += 1
21    return L
```

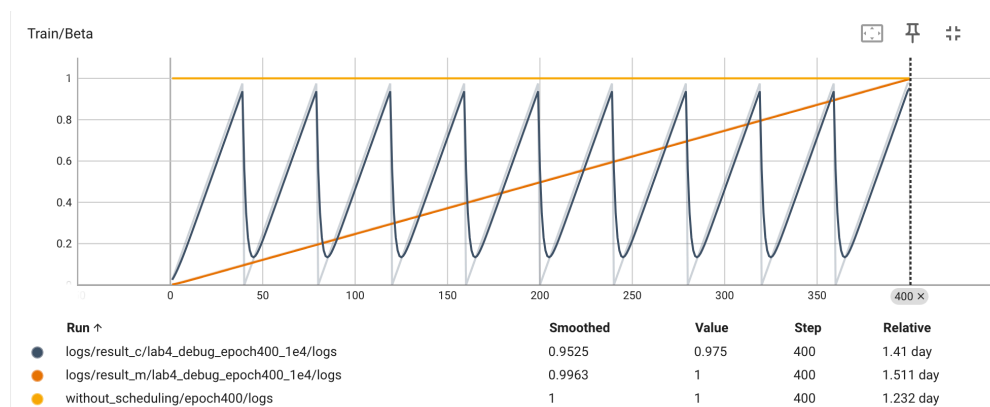


Figure 2: Different KL annealing ratio

3 Analysis & Discussion

3.1 Plot Teacher forcing ratio

這邊實驗嘗試了不同 Teacher forcing ratio 搭配 $\beta = 1$ ，比較它們之間的 train loss 下降差，由圖可知 (Figure 4, Figure 3) 不管 tfr 怎麼設定最後都能收斂到一個不錯的結果，差別在前期給比較多提示下，他在最後 validation 階段 PSNR 在同一個 epoch 下沒有那麼高，泛化能力沒有那麼好。

- 黃色：從 1 開始下降，每 10 步下降 0.1
- 紅色：完全不用
- 藍色從 1 開始下降，每 20 步下降 0.1

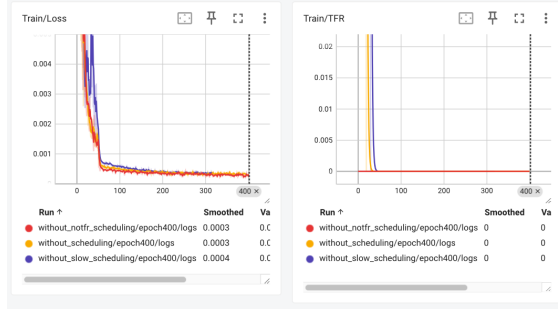


Figure 3: Train loss and TFR setting

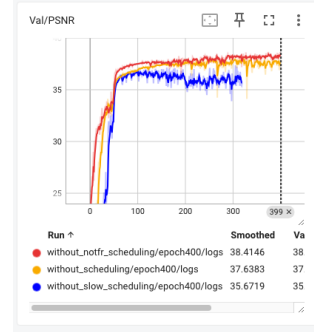
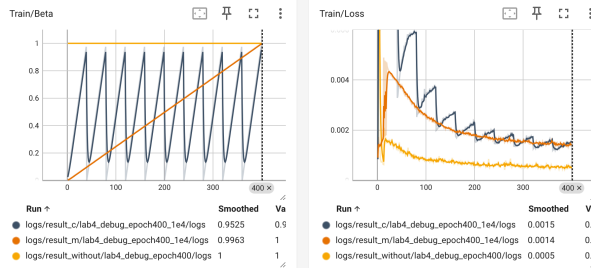


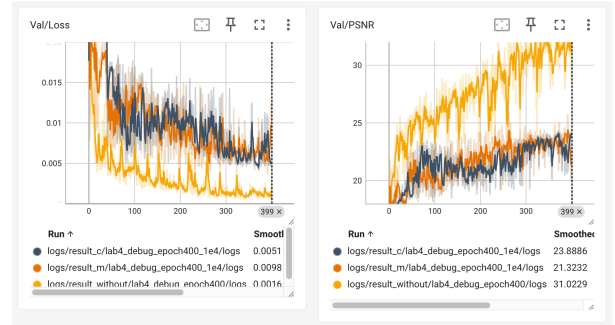
Figure 4: valid PSNR

3.2 Plot the loss curve while training with different settings

從實驗結果 (Figure 5a) 顯示 without 固定權重為 1 的表現結果最好，一開始三個的 loss 都會先劇烈上升再劇烈下降，最後都能收斂，但穩定的 KL loss 權重顯示有最好的訓練結果。Cyclical 的 loss 會隨著 β 降至 0 提高，提升至 1 又降低，顯示出 kl loss 權重為 1 的時後學習效果較好，在 validation 上也有相同的表現。



(a) KL loss comparison



(b) Validation loss and PSNR

Figure 5: Comparison between KL strategies and validation performance

3.3 Plot the PSNR-per-frame diagram in the validation dataset

由結果 (Table 1) 可知每個 frame 都有不錯的 PSNR 結果，彼此差異性不大平均都有在 35 以上。

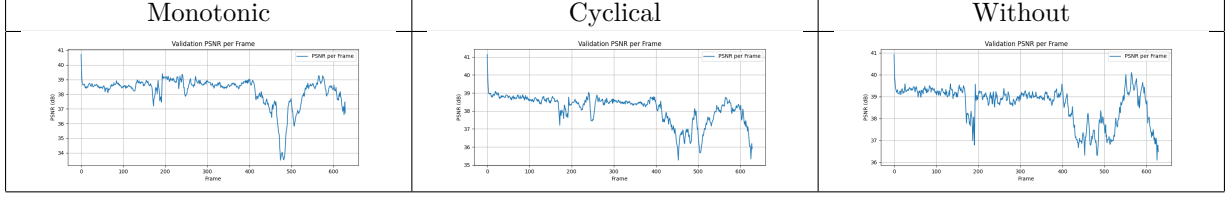


Table 1: PSNR per frame for different KL annealing strategies

3.4 Other training strategy analysis

嘗試了不同 learning rate scheduling 的方式，發現在一開始若有太大的 learning rate 容易導致梯度爆掉，loss 過高。為了防止這種狀況我們所有訓練設定起始都從 $1e4$ 開始下降。原始程式碼中的規則是在第 2 和第 5 個 epoch 將學習率乘上 0.1，使模型在早期快速學習後逐漸收斂，但訓練出來的效果不好，後來調整成每 50 個 epoch 將學習率乘上 0.1，並限制最小學習率不得低於 $1e-5$ ，用來控制長期訓練下的穩定性與避免學習率過低導致訓練停滯。下圖紅色會後來使用的 scheduling strategy，兩者都是建立在 Without($\beta = 1$) 的學習下。

```

1  # self.scheduler = optim.lr_scheduler.MultiStepLR(self.optim, milestones=[2, 5],
2      gamma=0.1)
3      self.min_lr = 1e-5
4      def lr_lambda(epoch):
5          factor = 0.1 ** (epoch // 50)
6          return max(factor, self.min_lr / self.args.lr)
7
8      self.scheduler = optim.lr_scheduler.LambdaLR(self.optim, lr_lambda=lr_lambda)

```

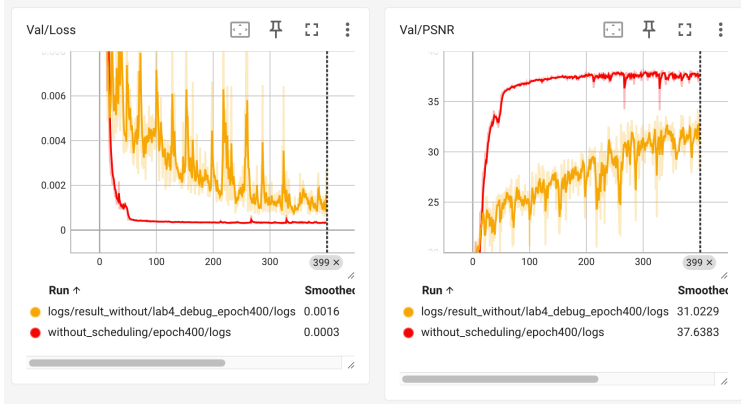


Figure 6: val loss and PSNR

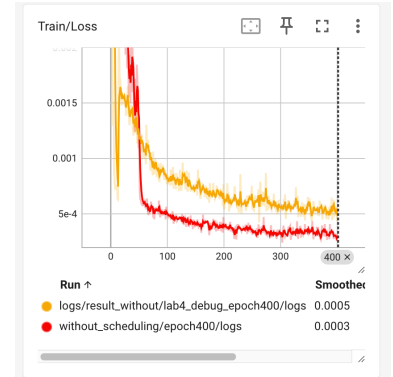


Figure 7: training loss